redirect is sent by our application to the browser, the browser sends a new request back to our application. By the time we receive that request, our application has moved on; all the instance variables from previous requests are long gone. The flash data is stored in the session to make it available between requests.

Armed with this background about flash data, we can create an `invalid_cart` method to report on the problem:

rails7/depot_h/app/controllers/carts_controller.rb

```
  class CartsController < ApplicationController
    before_action :set_cart, only: %i[ show edit update destroy ]
»   rescue_from ActiveRecord::RecordNotFound, with: :invalid_cart
    # GET /carts or /carts.json
    # ...
    private
    # ...

»     def invalid_cart
»       logger.error "Attempt to access invalid cart #{params[:id]}"
»       redirect_to store_index_url, notice: 'Invalid cart'
»     end
  end
```

The `rescue_from` clause intercepts the exception raised by `Cart.find`. In the handler, we do the following:

- Use the Rails logger to record the error. Every controller has a `logger` attribute. Here we use it to record a message at the `error` logging level.

- Redirect to the catalog display by using the `redirect_to` method. The `:notice` parameter specifies a message to be stored in the flash as a notice. Why redirect rather than display the catalog here? If we redirect, the user's browser will end up displaying the store URL rather than `http://.../cart/wibble`. We expose less of the application this way. We also prevent the user from retriggering the error by clicking the Reload button.

With this code in place, we can rerun our customer's problematic query by